AI INFRA RESEARCH SERIES

MoonEP

完美 token 均衡的理论条件、通信接口与训练代价

分析对象

MoonEP: A Perfectly Balanced Expert Parallelism Library via Dynamic Redundant Experts

作者与机构

Yutian Chen、Cong Li、Yucheng Wang、Ming Wei；Moonshot AI。

版本范围

资料类型为开源系统，而非冒充独立 arXiv 论文。依据官方仓库 d86fda40（2026-10-12）与 Kimi K3 v2 的 §5.2.1、附录 E。

核心判断

MoonEP 通过临时复制专家改变执行位置，使每个 EP rank 处理相同数量的真实路由 token。它不改变模型的逻辑专家选择，也不自动消除权重搬运、padding、梯度回收与不规则 GEMM 的成本。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 它到底是什么，与 MoK 有何不同 2

2 必须先统一的符号与假设 2

3 一个两卡例子：不改答案，改变计算地点 2

4 理论：为什么每卡最多 m 个冗余专家槽就足够 3

5 前向数据路径与接口所有权 4

6 反向：两类通信不能漏掉任何一类 4

7 Zero-copy 的收益与危险 5

8 静态形状、padding 与显存账本 5

9 什么时候复制专家才值得 6

10 基准审视：有哪些代价计入，哪些没有 6

11 语义正确、可重复与 batch invariance 7

12 复现路线与结论 7

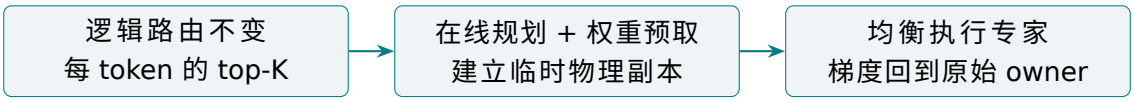
文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 它到底是什么，与 MoK 有何不同

MoonEP 是 Expert Parallelism 通信与运行时规划库。官方引用是 GitHub 项目；相应理论在 Kimi K3 报告中有明确说明，所以本报告把仓库实现与报告中的证明结合起来分析，不虚构独立论文编号。[S1–S2]

传统 EP 把专家固定分在多个 rank。token 由 router 选择专家后，需要去专家所在地计算；热门专家所在 rank 因此成为长尾。MoonEP 不要求 router 为了硬件均衡改选别的专家，而是让同一个逻辑专家临时在其他 rank 拥有副本。

MoK 主要优化给定 MoE 工作的计算通信融合；MoonEP 主要优化工作与专家副本的物理放置。它们关注不同层级。MoonEP 本身不负责 attention，也不是把整个 Expert FFN 融合成一个 Megakernel。[S1]



2 必须先统一的符号与假设

符号	含义
R	EP rank 数
S	每个 rank 路由前的输入 token 数
K	每个 token 选择的专家数
E	EP 域内的逻辑 routed expert 总数
m=E/R	每 rank 原始专家数；也作为远端预取槽数量
SK	每 rank 的目标真实 routed-token 工作量
NvS	加上每 VM group 对齐填充后的实际 token 槽位量

官方均衡叙述使用等 S、等 K、专家均匀分片且 E 可被 R 整除的设置。输入专家结构和数据类型也需满足库的接口约定。跨 rank 输入数量不同、专家计算大小不相同等情形，不能直接套用“每卡 SK 完美均衡”。[S1，Notation；S2，附录 E]

这里的 token 是**路由后的 token-expert assignment**。一个原 token 选 K 个专家，就贡献 K 份专家输入；这不意味着用户输入数量变成 K 倍。

静态容量也不是说每个专家的 M 永远相同。各专家收到的量仍然变化，GPU 上的累计偏移描述这些变长区间；固定的是可管理的 rank 级容量和预先安排的工作区。

3 一个两卡例子：不改答案，改变计算地点

教学配置：R=2、E=4、S=4、K=1。rank0 原始拥有专家 e0/e1，rank1 拥有 e2/e3。当前路由量是 $(6, 2, 0, 0)$，传统布局让 rank0 算 8 份，rank1 空闲。

将 e0 的权重复制到 rank1，让其中 4 份 e0 输入在副本上执行。此时 rank0 算 2 份 e0 + 2 份 e1，rank1 算 4 份 e0，双方各 4 份。原 token 仍由逻辑专家 e0 或 e1 处理，没有被改路由到不同的函数。

若两处 e0 参数相同，则实数数学中的函数值不变。反向时，两处 e0 的参数梯度相加，得到逻辑 e0 的完整梯度。副本不是独立专家，不应自己维护一套独立 optimizer 状态。

这个例子只是解释语义。真实系统还要恢复原 token 顺序、保留 top-K 权重、管理复制完成事件，并保证梯度回收后再更新参数。

4 理论：为什么每卡最多 m 个冗余专家槽就足够

4.1 优化对象是“冗余专家种类数”，不是延迟

K3 附录 E 定义，对于给定 router 输出 I 和放置计划 P，令 $m_r(P)$ 为 rank r 上的冗余专家数量，目标为

$$M(I) = \min_{P} \max_{r} m_r(P).$$

报告证明一个普遍上界：

$$M(I) \le \frac{E}{R} = m.$$

这是可行容量的保证，不等于线上 planner 对任意输入都算出全局最优通信延迟，更不等于没有复制成本。[S2，附录 E]

4.2 构造证明的直觉

最初每个 rank 执行它原始专家收到的全部任务。将 rank 分成过载和欠载两组。每次选择一对，从过载一方移出任务，把欠载一方填到 SK；接收方随后固定，不再修改。供给方可能仍过载、刚好平衡，或变成下一轮欠载方。

一个接收方只被填充一次，因此其远端任务可以来自同一个原始 owner group；这一组最多有 m 个逻辑专家。于是为该 rank 预留 m 个远端专家槽，就足以容纳这个构造需要的所有副本。最多经过 R−1 次填充达到全体均衡。[S2]

这里并不要求一次只移动整个专家。热门专家的 token 可以分段执行；正是允许拆分一个专家的工作，才能消除极端偏斜。

4.3 下界与“近似紧”的含义

报告还构造了 rank0 没有原生任务、其余 R−1 个 rank 上专家均分全部任务的情况。rank0 为了获得 SK 的工作，至少需要

$$\left\lceil \frac{E(R-1)}{R^2} \right\rceil$$

种远端专家。因此普遍上界 m 的数量级无法大幅降低。[S2，Theorem 2]

例如 E=256、R=8，上界为 32，下界构造给出 28。**这不是说所有现实 batch 必须复制 28–32 个专家**；它说明最坏情况下每卡所需槽位的理论规模。一般 batch 可能少得多。

4.4 证明没有承诺的东西

它不保证各专家 tile 数完全相等，不保证复制的权重字节最少，不保证所有 rank 的网络路径等价，也不保证 padding 后的实际计算时间相等。它平衡的是指定条件下的真实 token 工作数量。

因此，“perfect balance” 应该与 “不规则 GEMM 调度”“复制代价”“链路竞争” 分开看。K3 本身仍使用 workload-aware grouped GEMM 调度，正说明 rank 级均衡不是最终问题的结束。[S2，§5.2]

5 前向数据路径与接口所有权

5.1 dispatch 不只产生 tensor，还产生计划

官方 `Buffer.dispatch` 返回重排后的 token、对应路由权重、累计区间偏移 `cu_seqlens`，以及 `plan`。plan 必须保存，用于权重预取、combine 和反向的对应操作。[S1]

计算框架随后通过 `prefetch_weight`，把需要的 gate/up/down 专家投影填到临时槽中。Grouped GEMM 看到的是原始专家与本 rank 预取槽拼成的计算视图，而不是把所有 rank 的参数都复制到本地。

路由权重由接口携带，不应凭函数名猜测它在哪一步乘到 activation 上。接入框架要明确：权重是在激活融合、输出还是其他路径施加，避免重复乘权重或忘记乘权重。

5.2 两种视图，不能把虚拟映射当物理复制

每个投影的通信视图是 `[R,m,H,H']`，便于访问全体 rank 的预取池。当前 rank 的计算视图是 `[2m,H,H']`：前 m 行别名到本地参数，后 m 行别名到自己的预取池。[S1，Weight buffer]

从虚拟地址能看到 R 个 pool，不等于每张卡实际存了 R 份 pool。这里依赖对称／分布式缓冲与虚拟内存映射，算物理显存时必须按真正的本地分配计量。

`cu_seqlens` 描述 2m 个计算行的活跃 token 区间；有些区间可以为空。若错误假设所有槽位都启用或每个专家 M 相同，就会破坏结果或引入无效计算。

6 反向：两类通信不能漏掉任何一类

前向 combine 的反向，是把输出梯度按原计划重新分发给各专家执行位置；前向 dispatch 的反向，则把来自 K 个专家的输入梯度合并回原 token。[S1，API walkthrough]

此外还有 **临时专家副本的参数梯度回收**。副本梯度通过 reduce buffer 回到原始 owner，与 owner 本地贡献相加。它和框架在 EDP/DP 组中的梯度归约不是同一件事。

$$\nabla W_e = \nabla W_e^{\text{home}} + \sum_{r \in \text{replicas}(e)} \nabla W_{e,r}.$$

上式是相加，不应因为复制了 q 份就随意除以 q。后续 loss normalization、有效 token 数和数据并行平均应由框架正确处理；把 “副本合并” 和 “样本平均” 混在一起，会改变训练目标。

保存原 plan 也很重要：backward 必须沿与对应 forward 相容的映射返回，而不能重新用最新 batch 的路由做一次规划。

6.1 共享预取池意味着反向可能需要再预取

官方预取池跨层共享，降低常驻显存。但下一层可能覆盖上一层的副本权重。到上一层 backward 时，不能假设前向时那份远端权重仍在；基准代码的反向 dispatch 路径明确包含再次 prefetch。[S3]

同理，若训练框架在某个阶段更新了 owner 参数，而反向仍需要旧版本，应由严格的 step 与参数版本协议保证一致。共享临时存储是一种生命周期优化，不是让数据永远存在。

7 Zero-copy 的收益与危险

默认接口会返回新 tensor 或在通信边界进行拷贝；显式 `zero_copy=True` 才让 dispatch 返回通信 buffer 的视图，expert FFN 直接读写，再由 combine 消费。[S1，zero_copy]

这可以减少一次通信 buffer 到用户 buffer 的复制和对应 HBM 流量，但 ownership 更复杂：下一次 dispatch/combine 会覆盖同一内存。文档明确提醒，autograd 不能把这些别名视图作为长期 activation 保存到后续 backward。

可选方案是复制需要保存的激活、使用安全 stashing、重计算，或关闭 zero-copy。不同方案在显存与重算时间之间有不同代价。

因此，不能同时声称 “零拷贝”“任意跨层保存”“完全没有额外激活管理”，除非实现清楚证明了它们的生命周期并不冲突。

8 静态形状、padding 与显存账本

每 rank 有 SK 个真实 assignment，但每个 VM group 可能按 token_padding 对齐，因此实际 NvS 会大于 SK。**真实任务完美均衡，不等于没有 padding**。[S1]

假设 gate、up、down 三个投影都有 HH' 个元素，每 rank 有 m 个临时槽。仅 BF16 副本权重的主要额外本地存储为

$$M_{\text{replica,w}} = 3mHH' \times 2 \text{ bytes}.$$

FP32 副本梯度池相应为

$$M_{\text{replica,g}} = 3mHH' \times 4 \text{ bytes}.$$

教学估算：m=32、H=7168、H′=2048，分别约 **2.625 GiB 权重**与 **5.25 GiB 梯度**，尚未计通信 token buffer、对齐、plan 和框架其他内存。这不是作者实测，只说明“少量冗余专家”仍可能有可观的固定成本。

由于预取池跨层共享，不能再无条件乘以模型 MoE 层数；但共享也带来恢复与同步成本。部署时必须同时记录 peak allocated、reserved 和 VMM 相关分配，不能只看 PyTorch 一项统计。

9 什么时候复制专家才值得

设最慢 rank 因偏斜暴露的时间为 T_{tail}。启用 MoonEP 的额外成本包括 planner、权重搬运、梯度回收和管理。真正的收益条件是

$$\Delta T_{\text{tail}} > T_{\text{plan}} + T_{\text{prefetch,exposed}} + T_{\text{grad,exposed}} + T_{\text{manage}}.$$

这里使用 exposed 时间，因为能与独立计算重叠的部分不全部落在关键路径，但总带宽与 SM 消耗仍存在。大 H/H′、很小 token batch、低带宽网络，都可能使搬专家比搬 token 更贵。

如果路由本身几乎均匀，额外 planner 和复制池的价值也需要测量。不能用极端偏斜时的优势推断所有工作负载都更快。

10 基准审视：有哪些代价计入，哪些没有

官方 README 的主要图在 H20、EP8 上扫描路由偏斜，与 DeepEP v2 比较。偏斜指标是

$$\text{maxvio} = \max_e(T_e/\bar{T}) - 1.$$

它是**专家级**偏斜，不直接等于 rank 级偏斜；不同专家放置可以让相同 maxvio 产生不同的最慢 rank 时间。[S1]

固定代码 `bench_vs_deepep.py` 的优点是使用相同路由矩阵、输入、32 SM 预算与 128 对齐，并让 DeepEP 使用 expanded dispatch/reduced combine，对齐重排语义。MoonEP dispatch 计入 planning 与 weight prefetch。[S3，文件头及 time_op]

但必须同时知道：

基准细节	解释
grad_reduce 不计入	代码认为它可被后续计算覆盖，不在其选定 critical path
eager CUDA event、跨 rank mean	不等于完整训练 step 的最慢 rank wall time
固定预热与重复迭代	不包含首次编译、冷缓存等启动成本
单节点 EP 域	不能直接外推任意跨机网络
DeepEP 配置非 deterministic	不能据此证明同确定性约束下的所有相对结果

以上均来自本次固定提交的基准代码。[S3] “可以 overlap” 是需要完整执行依赖支持的前提，不是梯度回收成本天然消失。完整训练验收应在 optimizer step 前确认所有副本梯度真正完成，并测 wall time 和链路占用。

README 中 “高偏斜下训练不 OOM” 的描述，应理解为该测试配置的观察；它不保证任意模型、任何 batch、无限 host/device 资源需求都不 OOM。[S1]

11 语义正确、可重复与 batch invariance

只要临时副本权重一致、输出正确回收、梯度正确归并，实数数学上可以保持原逻辑 MoE 的函数。然而浮点归约次序可能随物理分配变化，不自动满足逐位一致。

应分别测试：相同 plan 的重复运行、不同物理复制计划、改变 EP rank 数、改变 batch 划分，以及 BF16/FP32 梯度传输。路由索引一致只是第一层；参数更新是否一致，还取决于归约、量化和优化器路径。

这也说明 MoonEP 与 MoK 的整合不能只关注吞吐。MoK 的固定归约结构若遇到动态副本梯度来源，必须重新明确顺序和 ownership，才有可能保留更强的确定性。

12 复现路线与结论

建议先跑官方 planning/dispatch/combine/prefetch/grad-reduce 测试，再用小模型核对完整 forward、dX、dW 与一次 optimizer update。用人为极端路由覆盖单专家热门、单 owner 热门、多个热点与空专家。

第二步独立测权重与梯度搬运，不先假设它们可隐藏。第三步在真实训练流水下启用 overlap，测最慢 rank、跨步残留通信、峰值显存与 p95 step 时间。zero-copy 测试必须包含至少两层交替执行和 backward，以暴露别名覆盖问题。

结论：MoonEP 将负载不均从 “让模型少选热门专家” 转成 “热门专家可以临时多处执行”。它的理论容量上界很漂亮，但最终速度由副本管理、GEMM 与通信共同决定。

[S1] 官方仓库与接口。 MoonEP README，固定提交 d86fda40。

[S2] 理论与系统上下文。 Kimi K3 v2，§5.2.1、附录 E。上界与下界证明是报告中的结论；本文给出的两卡演示与显存算式是解释性推导。

[S3] 性能计时源码。 benchmarks/bench_vs_deepep.py。

本报告未运行多 GPU 基准或训练，也不把工程 README 的宣传性措辞当作对所有负载的数学保证。